

map vs unordered_map — Խորը ուղեցույց

C++ հարցազրույցի ամբողջական presentation

Այս ուղեցույցը C++-ի associative container-ների խորը վերլուծություն է՝ `map` vs `unordered_map` (ներքին կառուցվածք `RB-tree` vs `hash table`, `complexity`, `collision`, `sorted` vs `unsorted`), բոլոր կարևոր ֆունկցիաներով, `find` vs `operator[]`, `lower_bound` / `upper_bound`, `typical problem`-ներով և `interview` հարցերով: Ներառված են նաև `set` / `unordered_set`:

1. Հիմնական տարբերությունը

map — Red-Black Tree (sorted)

```
map<int, string> mp;  
mp[3] = "C";  
mp[1] = "A";  
mp[2] = "B";  
  
for (auto& x : mp) cout << x.first << " "; // 1 2 3 (Միշտ sorted)
```

Ներսում` Balanced BST (Red-Black Tree): Key-երը միշտ պահվում են **sorted** կարգով:

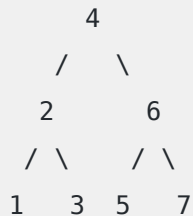
unordered_map — Hash Table (unsorted)

```
unordered_map<int, string> mp;  
mp[3] = "C";  
mp[1] = "A";  
mp[2] = "B";  
  
for (auto& x : mp) cout << x.first << " "; // 2 3 1 կամ gankacac karg
```

Ներսում` Hash Table: Key-երը **sorted չեն**, հերթականությամբ երաշխավորված չեն:

2. Ներքին կառուցվածք

map — Balanced Tree



Search: $O(\log n)$ (tree-ի բարձրություն)

Ամեն node ունի `key`, `value`, `parent`, `left`, `right`, `color` (red/black): Tree-ն ինքն իրեն balance է պահում insert/erase-ի ժամ:

unordered_map — Hash Table + Buckets

`hash(key) -> bucket index`

Bucket 0:

Bucket 1: -> (key=1)

Bucket 2: -> (key=2)

Bucket 3: -> (key=10) -> (key=20) <- collision

Key-ը անց է `hash(key)` ֆունկցիան-ի, ստանում bucket index, և ընկնում այդ bucket-ի մեջ.

3. Complexity

Գործողություն	map	unordered_map
insert	$O(\log n)$	$O(1)$ average, $O(n)$ worst
find	$O(\log n)$	$O(1)$ average, $O(n)$ worst
erase	$O(\log n)$	$O(1)$ average, $O(n)$ worst
ordered traversal	$O(n)$, sorted	Հնարավոր չէ (unsorted)
lower/upper_bound	$O(\log n)$	Չկա

Interview trap. **unordered_map** միշտ $O(1)$ է՞ -> ՈՉ: Average $O(1)$, worst $O(n)$ (չat collision-ի դեպքում):

4. Collision — unordered_map-ի հիմնական գաղափար

Երբ երկու տարբեր key ստանում են նույն hash/bucket.

```
hash(10) = 3  
hash(20) = 3      // նույն bucket -> COLLISION
```

Bucket 3:

```
|  
(10)  
|  
(20)  <- chaining (linked list bucket-ի ներքին)
```

Շատ collision -> bucket-ի list-երը երկարում -> lookup-ի արագություն $O(n)$ -ին -> performance-ի նվազում. Լավ hash function-ը collision-երը նվազեցնում է:

Load factor & rehashing

```
unordered_map<int,int> m;  
m.load_factor();           // size / bucket_count  
m.max_load_factor();       // sahmann (default ~1.0)  
m.reserve(1000);           // naxapes bucket-ner -> qich rehash
```

Երբ load factor-ը զերայանում է max-ը, container-ը **rehash** է անում (bucket-երը փոխարկում, բոլոր element-երը տեղափոխում) -> այդ պահին ժամկետ $O(n)$:

5. find vs operator[] (ՇԱՏ սիրված հարց)

```
map<int, string> mp;
mp[1] = "Arman";

cout << mp[100];           // key 100 ՉԿԱ -> ԻՆՔԸ ԻՐԵՆ ՍՏԵՂԾՈՒՄ Է (default value)!
                           // հիմա mp.size()-y mecacav
```

`operator[]` -ը՝ եթե key-ը չկա, **ստեղծում է** այն default value-ով: Դա հաճախ bug-i առնու է:

```
auto it = mp.find(100);    // չի ստեղծում
if (it != mp.end())
    cout << it->second;    // apahov
```

count() — գոյության ստուգում

```
if (mp.count(10))          // 0 կամ 1 (map-ում key-ն unique)
    cout << "Exists";
```

Կանոն. **կարճալ/ստուգել՝ find() կամ count(), ոչ operator[]** (որ պահանջում է չստեղծես key):

6. Map-ի հստակ ֆունկցիաներ (sorted-ի shnorhiv)

lower_bound / upper_bound

```
map<int,int> mp;  
mp[10]=1; mp[20]=2; mp[30]=3;  
  
auto it1 = mp.lower_bound(15);    // araji key >= 15 -> 20  
auto it2 = mp.upper_bound(20);    // araji key > 20 -> 30
```

- `lower_bound(k)` -> araji key $\geq k$
- `upper_bound(k)` -> araji key $> k$

Urancov anum tu range query-ner (or. «bolor key-ery [15, 25] mijakayqum»): **unordered_map-um sranc ՉԿԱՆ**, qani or sorted չէ:

7. Բոլոր կարևոր ֆունկցիաներ

Insert

```
mp.insert({1, "Arman"});
mp.insert(make_pair(2, "David"));
mp[3] = "Anna";           // kam operator[]
mp.emplace(4, "Nune");     // in-place
```

Access & search

```
mp[1];                     // access (steղcum e ete chka!)
mp.at(1);                  // bounds-checked (throw ete chka)
mp.find(10);               // iterator kam end()
mp.count(10);              // 0 kam 1
```

Remove & info

```
mp.erase(10);              // key-ov
mp.erase(mp.find(10));     // iterator-ov
mp.size();
mp.empty();
mp.clear();
```

Iteration

```
for (auto it = mp.begin(); it != mp.end(); ++it)
    cout << it->first << " " << it->second << "\n";

for (auto& p : mp)         // range-based
    cout << p.first << " " << p.second << "\n";
```

map-um iteration-ն sorted է; unordered_map-um`-ը կամայական կարգով.

8. Typical Problems (interview-i klasik)

Frequency Count (υλθενataracvacy)

```
vector<int> nums = {1,1,2,2,2,3};
unordered_map<int,int> freq;
for (int x : nums) freq[x]++;      // 1->2, 2->3, 3->1
// O(n) average (unordered); map-ov` O(n log n)
```

Duplicate Detection

```
unordered_set<int> seen;
for (int x : nums) {
    if (seen.count(x)) { cout << "Duplicate"; break; }
    seen.insert(x);
}
```

Character / Word Frequency

```
string s = "aabbccc";
unordered_map<char,int> cnt;
for (char c : s) cnt[c]++;        // a->2, b->2, c->3

unordered_map<string,int> wf;
for (string& w : words) wf[w]++;
```

Frequency/lookup ηυndirneri hamar ****unordered_map**** (O(n) total, O(1) average ηεκ element): Ete petq t sorted result map (O(n log n)):

9. set vs unordered_set

Հնչյնն unramabanuthyunny, bayc miayn key (arjeq chka):

```
set<int> s;           // Red-Black Tree, sorted,  $O(\log n)$ 
s.insert(5); s.insert(1); s.insert(3);
// iteration: 1 3 5 (sorted)
```

```
unordered_set<int> us; // Hash, unsorted,  $O(1)$  average
us.insert(5);
us.count(5);           // goyuthyun stugum
```

```
set          -> sorted,  $O(\log n)$ , lower_bound ka
unordered_set -> unsorted,  $O(1)$  avg, arag lookup
```

10. Երբ որն օգտագործել

map / set:

- + sorted order petq է
- + lower_bound / upper_bound / range query
- + inorder traversal
- ավելի ճանաչում $O(\log n)$

unordered_map / unordered_set:

- + ամենաարագ lookup/insert/erase ($O(1)$ average)
- + order կարելի է
- + frequency count, duplicate detection, caching
- worst case $O(n)$, sorted չէ, lower_bound չկա

DEFAULT: unordered_map արագ lookup-ի համար;

map`-ը օգտագործվում է, երբ անհրաժեշտ է range query կամ order

11. Interview հարցեր & պատասխաններ

map vs unordered_map հիմնական տարբերություն?

map = Red-Black Tree, sorted keys, $O(\log n)$. unordered_map = Hash Table, unsorted, $O(1)$ average / $O(n)$ worst.

Inch data structure en ogtagorcum?

map -> Red-Black Tree (balanced BST); unordered_map -> Hash Table (buckets + hashing).

unordered_map misht $O(1)$ e?

Voch. Average $O(1)$, worst $O(n)$ (shat hash collision kam rehashing).

Inch e collision-y?

Erku key nuyn bucket/hash unanalys; luzvum e chaining-ov (linked list bucket-um) kam open addressing-ov.

Inchu map-y sorted e?

Qani or Red-Black Tree-n inorder traversal-y talis e sorted kargov.

find vs operator[]?

operator[] key-y steghcum e ete chka (kox erevuyt); find() chi steghcum, apahov e kardaluc.

lower_bound vor container-um ka?

map/set-um (sorted). unordered_map/set-um CHKA.

Erb map, erb unordered_map?

unordered_map default arag lookup-i hamar; map erb sorted order / range query / lower_bound petq է.

at() vs operator[]?

at() bounds-checked (throw ete chka); operator[] steghcum e default value.

load factor / rehash?

load factor = size/buckets; gerazancelis rehash (buckets x2, verabashxum) -> ayd pahin $O(n)$.

Cheat Sheet

`map` (Red-Black Tree):

sorted | $O(\log n)$ all | lower/upper_bound | range query | more pointers/node

`unordered_map` (Hash Table):

unsorted | $O(1)$ avg, $O(n)$ worst | NO lower_bound | frequency/lookup | buckets

find vs []: find() safe (no create) | [] CREATES missing key (bug source)

count(): 0 or 1 (unique key) -> existence check

at(): bounds-checked (throws)

COLLISION: 2 keys -> same bucket -> chaining -> worst $O(n)$

LOAD FACTOR: size/buckets; exceed -> rehash ($O(n)$ that moment); reserve() helps

set / unordered_set: same but key-only (no value)

ONE-LINER:

"Use unordered_map for fast lookups; use map when you need ordered data, range queries, lower_bound(), or sorted traversal."